KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

TRAINING PROGRAM REGISTRATION AND PARTICIPANT TRACKING SYSTEM

Submitted by

NAME: NAVEEN KUMAR.K

REGISTER NO: 25BCS240

Under the Guidance of

MR. DHARANEESH

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: 2025 – 2026

APRIL 2026

Semester: II

DECLARATION

I hereby declare that the project entitled “TRAINING PROGRAM REGISTRATION AND PARTICIPANT TRACKING SYSTEM” submitted in partial fulfillment of the requirements for the course JAVA PROGRAMMING is a record of my work carried out under the guidance of MR. DHARANEESH, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

NAVEEN KUMAR.K

(Student)

CERTIFICATE

This is to certify that the project entitled “TRAINING PROGRAM REGISTRATION AND PARTICIPANT TRACKING SYSTEM” is a bona fide work carried out by NAVEEN KUMAR.K, II YEAR, B.Sc. COMPUTER SCIENCE, KG College of Arts & Science, in partial fulfillment of the requirements for the course JAVA PROGRAMMING.

This project has been completed under my supervision and guidance during the academic period 2025–2026.

Place: KGCAS

Date: _____

Submitted for the Viva-Voce Examination held on _____

Guide	**Head of the Department**
(Technical Trainer, KGCAS)	(Technical Head, KGCAS)
MR. DHARANEESH	MR. BOOPALAN

TABLE OF CONTENTS

1 Introduction
1.1 Objective
1.2 Overview
1.3 Features
2 Program Analysis
2.1 Problem Definition
2.2 Project Overview
2.3 Scopes and Limitations
3 System Design
3.1 Modules
4 Methodology
4.1 Problem Analysis
4.2 System Design
4.3 Program Development
4.4 Testing and Validation
4.5 Result and Documentation
4.6 Algorithm
4.7 Flowchart
5 Source Code
6 Input and Output Screens
7 Discussion and Conclusion
7.1 Discussion
7.2 Conclusion
8 References

ABSTRACT

The Training Program Registration and Participant Tracking System is a Java-based application designed to simplify the registration of participants for training programs and maintain basic participant records. Manual registration can cause duplicate entries, difficulty locating participant details, delays in counting registrations, and errors while tracking attendance.

This project provides a simple solution by displaying available training programs, accepting participant details, registering participants for a selected program, listing registered participants, and recording attendance status. The application assigns a registration number to each participant and displays a summary of registration and attendance information.

The application demonstrates Core Java concepts such as arrays, loops, conditional statements, methods, and keyboard input through a console-based interface. It is intended as an educational project and can later be extended with database storage, user authentication, and report generation.

The main objective is to reduce manual work, improve registration accuracy, and make basic participant tracking easier for training coordinators.

1. INTRODUCTION

Training programs are organized by colleges, institutions, and organizations to improve knowledge and practical skills. Coordinators need to collect participant details, assign participants to programs, and track attendance. When these tasks are handled manually, records may be misplaced, duplicate registrations may occur, and attendance information can become difficult to maintain.

The Training Program Registration and Participant Tracking System uses Java to organize training program information and automate basic registration operations. The user can view available programs, register participants, display registered participant details, and update attendance status.

1.1 Objective

- To develop a Java-based training registration and participant tracking application.
- To maintain participant names, contact details, selected programs, and registration IDs.
- To validate participant registration input and reduce duplicate or incomplete entries.
- To record and display participant attendance status.
- To reduce manual record keeping and save time for training coordinators.

1.2 Overview

The system is a menu-driven console application. It displays training programs and accepts participant details. The program validates the selected program and stores the participant record in memory. The coordinator can view registrations, mark attendance, and display a summary.

1.3 Features

- Training program list with program ID, title, and available seats.
- Participant registration with an automatically assigned registration ID.
- Input validation for participant name and selected program.
- Display of all registered participants.
- Attendance marking as Present or Absent.
- Registration summary and simple menu-based exit option.

2. PROGRAM ANALYSIS

Program analysis identifies problems in the existing registration process and defines the requirements for the proposed system. It helps determine the information to store, the operations to support, and the expected output.

2.1 Problem Definition

Manual training registration may result in duplicate entries, missing participant details, time-consuming searches, difficulty identifying the selected program, and inconsistent attendance records. The proposed Java application addresses these problems by organizing registration details and supporting basic attendance tracking.

2.2 Project Overview

- Displays the available training program catalogue.
- Accepts participant name and contact information.
- Allows the user to select a program by program ID.
- Validates the program selection and available seats.
- Assigns a registration ID and stores the participant record.
- Displays participant records and updates attendance status.

2.3 Scopes and Limitations

Scope: College workshops, technical training sessions, seminars, skill-development programs, and educational demonstrations of registration and attendance logic.

Limitations: This version stores data in program memory only; participant and attendance records are not saved after the program closes. It is console-based and does not include database connectivity, email notifications, online payment, user accounts, or multiple-coordinator support.

3. SYSTEM DESIGN

System design defines the structure of the application and the interaction between modules. The project is a menu-driven Java program with methods for displaying programs, registering participants, listing registrations, recording attendance, and showing a summary.

3.1 Modules

1. Training Program Display Module

Displays the program ID, title, and available seats.

ID	Training Program	Available Seats
1	Java Programming	30
2	Web Development	25
3	Python Basics	20

2. Participant Registration Module

Accepts participant name, contact details, and selected program ID. It validates the input and creates a registration record.

3. Participant Tracking Module

Stores participant details in the current session and displays registered participants with their registration IDs and selected programs.

4. Attendance Module

Allows the coordinator to mark a participant as Present or Absent and displays the current attendance status.

5. Summary and Exit Module

Displays the number of registrations and attendance counts, and allows the coordinator to end the application safely.

4. METHODOLOGY

The methodology describes the stages followed to develop the project, from understanding the problem to designing, implementing, testing, and documenting the system.

- Problem Analysis
- System Design
- Program Development
- Testing and Validation
- Result and Documentation

4.1 Problem Analysis

The manual training registration process was examined to identify tasks that could be automated.

- Handwritten forms may contain incomplete or unclear information.
- Finding a participant record can take time.
- Duplicate registrations may be difficult to identify.
- Attendance records may not be updated consistently.
- Counting registrations and attendance manually increases the coordinator’s workload.

Based on these problems, the project requirements were defined as program display, participant registration, record listing, attendance updating, and summary generation.

4.2 SYSTEM DESIGN

The application is organized into small methods to make the code easier to understand and maintain.

- Program Module – stores and displays training program information.
- Registration Module – receives participant details and selected program.
- Validation Module – checks program IDs and input values.
- Tracking Module – lists participant records.
- Attendance Module – updates attendance status.
- Summary Module – displays registration and attendance totals.

Input/output sequence:

1. Start the application.
2. Display the main menu and training programs.
3. Read the coordinator’s menu choice.
4. Collect participant details when registration is selected.
5. Validate the program ID and available seats.
6. Assign a registration ID and store the participant.
7. Allow the coordinator to list participants or mark attendance.
8. Display the summary and end the program when requested.

4.3 PROGRAM DEVELOPMENT

Technologies Used

- Java (JDK 8 or above).
- IDE: Eclipse, IntelliJ IDEA, or Visual Studio Code.
- Console input/output using the Scanner class.

Core Java Concepts Used

- Arrays to store participant names, contact details, program IDs, and attendance.
- Loops to display programs and repeat the menu.
- Conditional statements to validate choices and attendance options.
- Methods to separate registration, display, attendance, and summary tasks.
- Arithmetic operations to count registrations and attendance totals.

Development Steps

- Created the main Java class and training program data.
- Implemented the training program catalogue display.
- Added input handling for participant details.
- Implemented registration ID assignment and seat validation.
- Added participant listing and attendance update options.
- Created summary output and tested the program.

4.4 TESTING AND VALIDATION

Testing checks whether the application displays training programs correctly, accepts valid participant details, assigns registration IDs, records attendance, and handles invalid input.

Testing Done

- Verified program titles and available seat values.
- Checked registration using valid participant details.
- Checked invalid program IDs and empty participant names.
- Verified participant listing and registration ID display.
- Tested Present and Absent attendance updates.
- Checked summary counts and the exit option.

Test Case	Input / Condition	Expected Result
Valid registration	Name and program ID 1	Participant registered and ID displayed
List participants	Select list option	Registered participant details displayed
Invalid program	Program ID 9	Error message displayed
Attendance update	Valid registration ID; Present	Attendance status updated
Invalid registration ID	Unknown ID	Error message displayed
Exit	Select exit option	Program ends safely

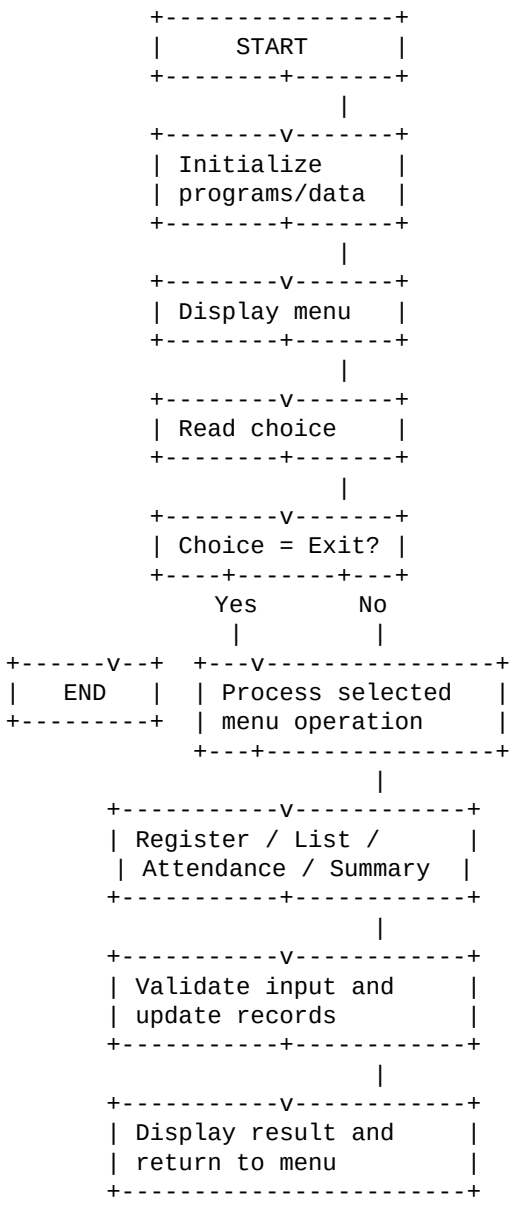
4.5 RESULT AND DOCUMENTATION

After implementing and testing the application, it was able to display training program details, register participants, list registration records, update attendance, and generate a simple summary. The documentation explains project objectives, requirements, modules, methodology, algorithm, source code, sample output, and conclusion.

4.6 ALGORITHM

9. Start the program.
10. Initialize training program names and seat counts.
11. Display the main menu.
12. Read the coordinator’s choice.
13. If registration is selected, read participant details and program ID.
14. Validate the details and check seat availability.
15. If valid, assign a registration ID, store the participant, and reduce available seats.
16. If participant listing is selected, display all stored records.
17. If attendance is selected, read the registration ID and attendance status.
18. Validate the registration ID and update attendance.
19. Display the summary when requested.
20. Repeat until Exit is selected, then end the program.

4.7 FLOWCHART



5. SOURCE CODE

This Core Java program provides a console-based training program registration and participant tracking workflow. The sample catalogue contains Java Programming, Web Development, and Python Basics. Participant records are stored in memory for the current program session.

```
import java.util.Scanner;\n\npublic class TrainingProgramSystem {\n    static Scanner sc = new
Scanner(System.in);\n    static String[] programs = {\n        "Java Programming", "Web Development",
"Python Basics"\n    };\n    static int[] seats = {30, 25, 20};\n    static String[] names = new
String[100];\n    static String[] contacts = new String[100];\n    static int[] programIds = new
int[100];\n    static String[] attendance = new String[100];\n    static int count = 0;\n    static
int nextRegId = 1001;\n\n    public static void main(String[] args) {\n        int choice;\n
do {\n            System.out.println("\\n===== TRAINING PROGRAM SYSTEM =====");\n
System.out.println("1. View Training Programs");\n            System.out.println("2. Register
Participant");\n            System.out.println("3. View Participants");\n
System.out.println("4. Mark Attendance");\n            System.out.println("5. View Summary");\n
System.out.println("0. Exit");\n            System.out.print("Enter choice: ");\n            choice =
sc.nextInt();\n            sc.nextLine();\n\n            switch (choice) {\n                case 1:
displayPrograms(); break;\n                case 2: registerParticipant(); break;\n
case 3: displayParticipants(); break;\n                case 4: markAttendance(); break;\n
case 5: displaySummary(); break;\n                case 0: System.out.println("Thank you!"); break;\n
default: System.out.println("Invalid choice!");\n            }\n        } while (choice != 0);\n
sc.close();\n    }\n\n    static void displayPrograms() {\n        System.out.println("\\nAvailable
Training Programs:");\n        for (int i = 0; i < programs.length; i++) {\n
System.out.println((i + 1) + ". " + programs[i]\n                + " | Seats available: " +
seats[i]);\n        }\n    }\n\n    static void registerParticipant() {\n        if (count >=
names.length) {\n            System.out.println("Registration capacity is full.");\n
return;\n        }\n        System.out.print("Enter participant name: ");\n        String name =
sc.nextLine().trim();\n        if (name.isEmpty()) {\n            System.out.println("Name cannot be
empty.");\n            return;\n        }\n        System.out.print("Enter contact number: ");\n
String contact = sc.nextLine().trim();\n        displayPrograms();\n        System.out.print("Select
program ID: ");\n        int id = sc.nextInt();\n        sc.nextLine();\n        if (id < 1 || id >
programs.length) {\n            System.out.println("Invalid program ID!");\n            return;\n
}\n        if (seats[id - 1] <= 0) {\n            System.out.println("No seats available.");\n
return;\n        }\n        names[count] = name;\n        contacts[count] = contact;\n
programIds[count] = id;\n        attendance[count] = "Not Marked";\n
System.out.println("Registration successful. Registration ID: "\n            + nextRegId);\n
nextRegId++;\n        seats[id - 1]--;\n        count++;\n    }\n
```

5. SOURCE CODE (CONTINUED)

```
    static void displayParticipants() {\n        System.out.println("\\nRegistered Participants:");\n
if (count == 0) {\n            System.out.println("No registrations found.");\n            return;\n
}\n        for (int i = 0; i < count; i++) {\n            System.out.println("Registration ID: " +
(1001 + i)\n                + " | Name: " + names[i]\n                + " | Contact: " + contacts[i]
\n                + " | Program: " + programs[programIds[i] - 1]\n                + " | Attendance: "
+ attendance[i]);\n        }\n    }\n\n    static void markAttendance() {\n        if (count == 0)
{\n            System.out.println("No participants registered.");\n            return;\n        }\n
System.out.print("Enter registration ID: ");\n        int regId = sc.nextInt();\n
sc.nextLine();\n        int index = regId - 1001;\n        if (index < 0 || index >= count) {\n
System.out.println("Invalid registration ID!");\n            return;\n        }\n
System.out.print("Enter attendance (Present/Absent): ");\n        String status =
sc.nextLine().trim();\n        if (status.equalsIgnoreCase("Present")) {\n
attendance[index] = "Present";\n        } else if (status.equalsIgnoreCase("Absent")) {\n
attendance[index] = "Absent";\n        } else {\n            System.out.println("Enter Present or
Absent only.");\n            return;\n        }\n        System.out.println("Attendance updated
successfully.");\n    }\n\n    static void displaySummary() {\n        int present = 0, absent = 0,
notMarked = 0;\n        for (int i = 0; i < count; i++) {\n            if
(attendance[i].equals("Present")) present++;\n            else if (attendance[i].equals("Absent"))
absent++;\n            else notMarked++;\n        }\n        System.out.println("\\n=====
REGISTRATION SUMMARY =====");\n        System.out.println("Total registrations: " + count);\n
System.out.println("Present: " + present);\n        System.out.println("Absent: " + absent);\n
System.out.println("Not marked: " + notMarked);\n        for (int i = 0; i < programs.length; i++)
{\n            System.out.println(programs[i] + " - Seats left: " + seats[i]);\n        }\n    }\n}\n
```

6. INPUT AND OUTPUT SCREENS

Sample Console Input and Output

```
===== TRAINING PROGRAM SYSTEM =====\n1. View Training Programs\n2. Register Participant\n3. View Participants\n4. Mark Attendance\n5. View Summary\n0. Exit\nEnter choice: 1\n\nAvailable Training Programs:\n1. Java Programming | Seats available: 30\n2. Web Development | Seats available: 25\n3. Python Basics | Seats available: 20\n\nEnter choice: 2\nEnter participant name: Naveen\nEnter contact number: 9876543210\nSelect program ID: 1\nRegistration successful. Registration ID: 1001\n\nEnter choice: 4\nEnter registration ID: 1001\nEnter attendance (Present/Absent): Present\nAttendance updated successfully.\n\nEnter choice: 5\n===== REGISTRATION SUMMARY =====\nTotal registrations: 1\nPresent: 1\nAbsent: 0\nNot marked: 0\nJava Programming - Seats left: 29\nWeb Development - Seats left: 25\nPython Basics - Seats left: 20\n\nEnter choice: 0\nThank you!\n
```

The example demonstrates viewing available programs, registering a participant, marking attendance, and displaying the registration summary. The Java Programming seat count decreases after a successful registration, and the participant’s attendance status is shown in the summary.

7. DISCUSSION AND CONCLUSION

7.1 Discussion

The Training Program Registration and Participant Tracking System was developed to demonstrate how Java can automate basic training coordination tasks. The application allows the coordinator to view training programs, register participants, list participant records, mark attendance, and display a summary.

Core Java concepts such as arrays, loops, conditional statements, methods, formatted output, and the Scanner class are used in the implementation. Testing with valid and invalid inputs helps confirm that the system validates program selection, assigns registration IDs, updates available seats, and records attendance status.

The current version keeps data in memory, so participant and attendance records are lost when the program closes. Future improvements could include database connectivity, permanent registration records, participant search and editing, email notifications, downloadable attendance reports, user login, and a graphical user interface.

7.2 Conclusion

The project demonstrates a simple Java-based solution for training program registration and participant tracking. It reduces manual record keeping, organizes participant details, supports attendance updates, and provides a summary of registrations.

The system meets the basic educational objectives and provides a foundation for a more advanced training management application.

8. REFERENCES

Books

- Herbert Schildt – Java: The Complete Reference.
- Kathy Sierra and Bert Bates – Head First Java.

Websites

- Oracle Java Documentation: https://docs.oracle.com/javase/8/docs/
- GeeksforGeeks Java: https://www.geeksforgeeks.org/java/
- W3Schools Java Tutorial: https://www.w3schools.com/java/
- Javatpoint Java Tutorial: https://www.javatpoint.com/java-tutorial